

A Greedy Exchange Method for Warehouse Slotting (GEMWS)

Muzamil Saiq

October 1, 2026

Abstract

This note presents a Greedy Exchange Method for Warehouse Slotting (GEMWS). The method matches volume partitions and repeatedly exchanges the highest-velocity rear-pick SKU with the lowest-velocity forward-pick SKU whenever the exchange increases total forward-pick velocity. Under the stated feasibility assumptions, the procedure terminates with an allocation that maximizes total forward-pick velocity within the selected pair of partitions.

1 Introduction

Forward-pick locations serve frequently picked SKUs, while rear-pick locations hold replenishment stock and lower-velocity picks. With a limited number of forward-pick locations, the assignment problem is to determine which SKUs should occupy them, subject to storage constraints.

We consider exchanges between existing locations rather than changes to the warehouse layout. The objective is to maximize total forward-pick velocity while preserving the number of assignments in each selected partition.

2 Related Work

Aase and Petersen (2022) describe a pairwise exchange heuristic for warehouse re-slotting. Their method exchanges higher-activity SKUs in the rear section with lower-activity SKUs in the front section, using ABC activity classes to determine whether an exchange is beneficial.

GEMWS uses a closely related exchange strategy, comparing numerical velocities within matched volume partitions. This note focuses on termination and optimality for total forward-pick velocity under explicit feasibility assumptions. Aase and Petersen instead evaluate picker travel through simulation; the optimality result presented here does not extend to that travel-distance objective.

For broader background on greedy and exchange methods, see Edmonds (1971) and Kernighan and Lin (1970), respectively. These references provide conceptual context rather than proofs of the warehouse-specific results presented here.

3 Assumptions

Each row represents a SKU assignment with a velocity and the capacity of its assigned storage bin. Each physical location has a fixed address and storage capacity. Sorting preserves these

associations. Exchanging SKUs changes their assigned locations without changing the locations' addresses or capacities.

Storage-bin capacities are grouped into discrete classes for packaging and allocation purposes. These classes represent volumetric capacity rather than detailed three-dimensional packing geometry.

Within each matched pair of partitions, every cross-boundary exchange considered by the method is feasible in both directions: the incoming SKU fits the forward-pick location, and the displaced SKU fits the rear-pick location. Each location holds one modeled assignment, and the number of assignments in each partition remains fixed.

Velocities remain constant during optimization. All locations within the selected forward-pick partition contribute equally to the objective. Relocation costs are excluded.

4 Volume Optimization

4.1 Defining Forward Pick FP & Rear Pick matrices RP

For both forward-pick and rear-pick matrices, if SKUs are not unique, we include an immutable address vector to distinguish entries by physical location. Each address remains attached to its corresponding row during sorting and partitioning.

$$\text{FP} = \begin{bmatrix} \text{SKU}_{F,1} & \text{Velocity}_{F,1} & \text{Volume}_{F,1} \\ \text{SKU}_{F,2} & \text{Velocity}_{F,2} & \text{Volume}_{F,2} \\ \vdots & \vdots & \vdots \\ \text{SKU}_{F,n} & \text{Velocity}_{F,n} & \text{Volume}_{F,n} \end{bmatrix},$$

$$\text{RP} = \begin{bmatrix} \text{SKU}_{R,1} & \text{Velocity}_{R,1} & \text{Volume}_{R,1} \\ \text{SKU}_{R,2} & \text{Velocity}_{R,2} & \text{Volume}_{R,2} \\ \vdots & \vdots & \vdots \\ \text{SKU}_{R,m} & \text{Velocity}_{R,m} & \text{Volume}_{R,m} \end{bmatrix}.$$

Here we assume that our volumes are binned, not granular, for future packaging considerations.

4.2 Sort by Volume

Sort rows by ascending volume, keeping SKU, velocity, and volume together:

$$\begin{aligned} \text{RP} &\leftarrow \text{SortRows}_{\text{Volume}\uparrow}(\text{RP}), \\ \text{FP} &\leftarrow \text{SortRows}_{\text{Volume}\uparrow}(\text{FP}). \end{aligned}$$

The distinct partition volumes are ordered as

$$v_{\text{RP},1} < \cdots < v_{\text{RP},L}, \quad v_{\text{FP},1} < \cdots < v_{\text{FP},K}.$$

4.3 Define Whole Partition Submatrices

Each partition contains every row with the corresponding volume, retaining all three columns: SKU, velocity, and volume.

$$V_{\text{RP},\ell} := \text{RP}[\{r : \text{Volume}_{R,r} = v_{\text{RP},\ell}\}, :],$$

$$\ell = 1, \dots, L.$$

$$V_{\text{FP},k} := \text{FP}[\{r : \text{Volume}_{F,r} = v_{\text{FP},k}\}, :],$$

$$k = 1, \dots, K.$$

4.4 Select the Forward-Pick Partition

Take a rear-pick partition $V_{\text{RP},\ell}$ in ascending order of volume and define

$$j := \min\{k : v_{\text{FP},k} \geq v_{\text{RP},\ell}\}.$$

If no such k exists, GEMWS considers no forward-pick partition for $V_{\text{RP},\ell}$, and its SKUs remain in rear pick.

Select the entire partition submatrix:

$$V_{\text{FP},j} = \text{FP}[\{r : \text{Volume}_{F,r} = v_{\text{FP},j}\}, :].$$

We show that $V_{\text{FP},j}$ is volumetrically optimal, meaning that it minimizes $v_{\text{FP},k}$ subject to $v_{\text{FP},k} \geq v_{\text{RP},\ell}$.

By definition of j , $v_{\text{FP},j} \geq v_{\text{RP},\ell}$, so $V_{\text{FP},j}$ is feasible.

Suppose a feasible partition $V_{\text{FP},k}$ has a smaller volume than $V_{\text{FP},j}$. Since the partition volumes are strictly increasing, $k < j$. But $V_{\text{FP},k}$ is feasible, so $v_{\text{FP},k} \geq v_{\text{RP},\ell}$ with $k < j$, contradicting the minimality of j .

Hence, $V_{\text{FP},j}$ is the feasible partition with the smallest volume and is therefore volumetrically optimal.

5 The Slotting Algorithm

For the selected volume-compatible partition submatrices, use velocity to determine the swaps. Retain volume as location metadata, but omit it from the velocity comparison.

Sort forward pick by descending velocity and rear pick by ascending velocity:

$$V_{\text{FP},j} \leftarrow \text{SortRows}_{\text{Velocity} \downarrow}(V_{\text{FP},j}),$$

$$V_{\text{RP},\ell} \leftarrow \text{SortRows}_{\text{Velocity} \uparrow}(V_{\text{RP},\ell}).$$

During sorting, each row's SKU, velocity, and location address remain together. During a physical swap, the SKU and its velocity change locations; each address continues to identify its fixed physical location.

Let the current sorted velocities be

$$f_1 \geq f_2 \geq \dots \geq f_N, \quad r_1 \leq r_2 \leq \dots \leq r_M.$$

If either matrix is empty, no swap is possible. Otherwise, repeat the following procedure.

5.1 Compare Across the Boundary

Compare the highest rear-pick velocity with the lowest forward-pick velocity:

$$\Delta := r_M - f_N.$$

If $\Delta \leq 0$, stop. Otherwise, swap the products assigned to those two locations.

Writing $P(F_N)$ and $P(R_M)$ for their SKU and velocity pairs:

$$(P(F_N), P(R_M)) \leftarrow (P(R_M), P(F_N)).$$

5.2 Restore the Sorted Order

Reinsert the incoming SKU into its sorted position in each matrix:

$$\begin{aligned} V_{FP,j} &\leftarrow \text{SortRows}_{\text{Velocity}\downarrow}(V_{FP,j}), \\ V_{RP,\ell} &\leftarrow \text{SortRows}_{\text{Velocity}\uparrow}(V_{RP,\ell}). \end{aligned}$$

Only the incoming row needs repositioning in each matrix. Retain every row and repeat the boundary comparison using the updated ordering.

5.3 Termination

Each swap strictly increases the total forward-pick velocity:

$$\Phi := \sum_{h=1}^N f_h, \quad \Phi_{\text{new}} - \Phi_{\text{old}} = r_M - f_N > 0.$$

There are finitely many allocations of the SKUs between the selected locations. Therefore, the procedure terminates.

5.4 Optimality Within the Selected Partitions

If either partition is empty, the allocation is trivially optimal within the selected pair because no cross-boundary exchange is possible. Otherwise, $N, M > 0$, and at termination,

$$r_M \leq f_N.$$

Since the matrices are sorted, every rear-pick velocity is no greater than every forward-pick velocity:

$$r_h \leq r_M \leq f_N \leq f_k, \quad 1 \leq h \leq M, \quad 1 \leq k \leq N.$$

Thus, forward pick contains the N highest velocities across the two selected partitions, allowing ties. Assuming every cross-boundary swap under consideration is feasible, this maximizes the total forward-pick velocity for these partitions.

6 Limitations

The optimality result applies to the selected pair of partitions under the stated feasibility assumptions. It does not establish global optimality across all warehouse partitions or directly minimize travel distance.

In practice, a displaced SKU may exceed the capacity of its proposed rear-pick location. Such an exchange falls outside the model’s assumptions. Accounting for location-specific capacities, packing geometry, and relocation costs requires an extended formulation, potentially using mixed-integer linear programming.

7 Conclusion

Under the stated feasibility assumptions, greedy exchanges maximize total forward-pick velocity within a matched pair of volume partitions. Each exchange strictly improves the objective, and the procedure terminates when the highest rear-pick velocity does not exceed the lowest forward-pick velocity. Extending this result to location-specific constraints and interactions across multiple partitions remains outside the scope of this note.

References

- Aase, G. R., & Petersen, C. G. (2022). A decision support system for re-slotting a case pick distribution center. *Open Journal of Business and Management*, 10(4), 1923–1935. <https://doi.org/10.4236/ojbm.2022.104099>
- Edmonds, J. (1971). Matroids and the greedy algorithm. *Mathematical Programming*, 1(1), 127–136. <https://doi.org/10.1007/BF01584082>
- Kernighan, B. W., & Lin, S. (1970). An efficient heuristic procedure for partitioning graphs. *Bell System Technical Journal*, 49(2), 291–307. <https://doi.org/10.1002/j.1538-7305.1970.tb01770.x>